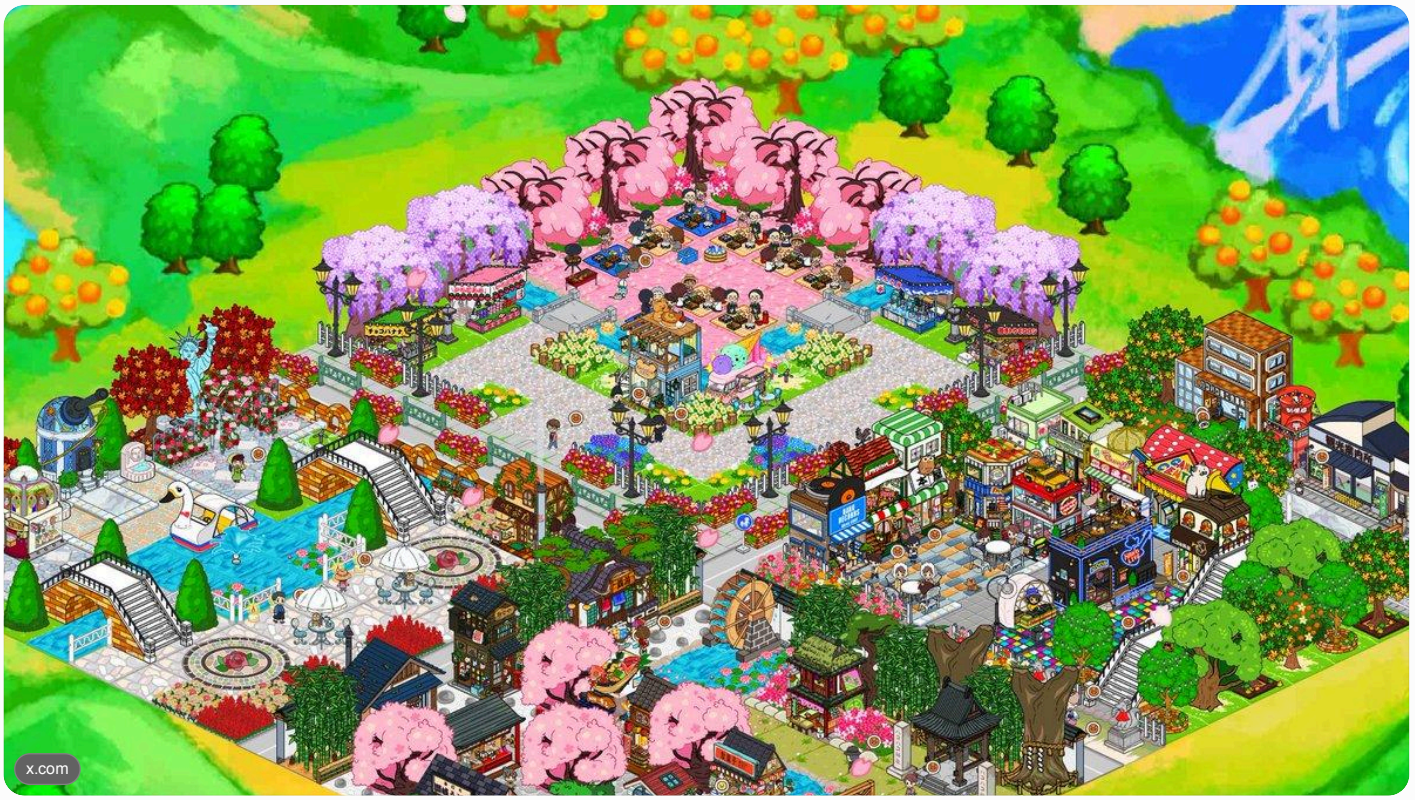
ChatGPT 說：

PixelNook 正式城市系統規格書 v1.0

設計日期：2026/11/01｜版本：City Life Expansion｜適用於 Claude Code 開發

這次的三個決策已經確定：

PixelNook City Life



等角移動
2:1 像素城市，點擊地板移動

混合打工
小遊戲、委託、簡單工作

分區城市
透過道路和入口前往不同區域

我先說明這次規格最重要的設計原則：

城市是遊戲的主要活動場所，房間是玩家的私人基地。

玩家不是單純進城市領錢，而是要在城市裡生活，有工作、有認識的人、有探索目標、有自己的收藏。

我會以你上傳的 v0.1 專案簡報為技術基礎，不會要求你重新使用 Unity，也不會直接推翻目前的程式。

目前已經存在的系統包括：

- HTML5 Canvas、原生 JavaScript。
- 64×32 px 等角地板格子。
- 800×480 邏輯畫布。
- 角色四方向移動與 BFS 尋路。
- 家具碰撞、場景排序。
- Firebase Realtime Database REST。
- 49 件家具、四種家具系列。
- 房間參觀、按讚與自動存檔。

這些都是城市系統可以繼續沿用的基礎。

像素小窩_專案簡報_20261101.pdf

第一章：遊戲整體架構

1.1 遊戲世界結構

PixelNook 未來分成兩個主要世界：

城市 City 主要玩法

玩家在這裡移動、工作、探索、與 NPC 互動。

- 城市地圖
- 建築與商店
- NPC
- 打工系統
- 委託任務
- 收藏與探索

私人房間 My Room 個人基地

玩家在這裡消費、裝飾、整理收藏。

- 家具佈置
- 服裝更換
- 背包管理
- 收藏展示
- 參觀其他玩家

兩個世界使用同一個玩家資料。

例如：

玩家在咖啡廳工作賺到 100 金幣，回到房間購買 80 金幣的家具，剩下 20 金幣。

這兩個場景不能各自維護不同的金幣資料，必須共用同一個經濟系統。

1.2 主要遊戲循環

純文字

```
登入遊戲
    │
    ▼
查看今日任務
    │
    ▼
選擇今天的活動
    │
    ├── 城市探索
    │       └── 發現物品、解鎖收藏
    │
    ├── NPC 委託
    │       └── 完成任務、獲得獎勵
    │
    └── 城市打工
            └── 完成工作、獲得薪水
    │
    ▼
獲得金幣、材料、收藏品
    │
    ▼
回到自己的房間
    │
    ▼
購買家具、佈置、整理收藏
    │
    ▼
參觀其他玩家的房間
    │
    ▼
```

這套循環是所有新系統的設計依據。

第二章：城市地圖系統

2.1 城市採分區式設計

城市不是一張無限延伸的大地圖，而是由多個獨立場景組成。

玩家可以在區域內自由移動，透過道路出口或建築入口切換場景。



我建議正式城市分成六個區域。

區域	主要建築	核心活動	開放階段
中央廣場	公告欄、噴水池	NPC、任務、活動	第一階段
商業街	咖啡廳、麵包店、超市	打工、購物	第一階段
城市公園	花園、池塘	探索、採集	第一階段
住宅區	公寓、住宅	NPC 劇情、房間入口	第二階段
海濱區	碼頭、魚市場	釣魚、收集、打工	第三階段
工坊區	木工坊、工作室	製作、材料加工	第三階段

第一版只需要完成前三個區域。

其餘區域可以先以封閉道路、施工圍欄或尚未開放的入口呈現。

這樣玩家知道城市未來還會擴大，也能降低初期開發量。

2.2 地圖尺寸規格

建議第一版採用以下數值：

項目	規格
等角格子	64×32 px
城市邏輯畫布	800×480 px
單一區域尺寸	24×18 格
單一區域地圖	1536×576 px 的等角平面範圍
可視範圍	800×480 px
地圖移動	攝影機跟隨角色

玩家移動	點擊地面、BFS 尋路
方向	左下、右下、右上、左上
建築	固定座標放置
NPC	指定活動範圍

這裡的 24×18 是地圖邏輯格數，不代表所有地板都必須完整繪製。

可以使用道路、草地、建築、裝飾等不同 Tile。

2.3 場景切換規則

例如：

純文字

```
中央廣場
  ├── 商業街
  ├── 城市公園
  └── 住宅區（未開放）

商業街
  ├── 中央廣場
  ├── 咖啡廳（室內）
  ├── 麵包店（室內）
  └── 超市（室內）

城市公園
  ├── 中央廣場
  └── 池塘（探索場景）
```

場景切換時：

1. 儲存玩家目前位置。
2. 讀取下一個場景資料。
3. 把角色放到對應出口。
4. 更新攝影機位置。
5. 載入該區域的 NPC。
6. 恢復角色移動與互動。

玩家進入建築後，也使用相同的場景管理方式。

2.4 城市場景資料格式

建議採用資料驅動設計，不要把所有建築座標寫死在遊戲主程式。

JavaScript

```
const cityAreas = {
  central: {
    id: "central",
    name: "中央廣場",
    width: 24,
    height: 18,

    exits: [
      {
        direction: "south",
        target: "commercial",
        spawn: { x: 12, y: 2 }
      },
      {
        direction: "east",
        target: "park",
        spawn: { x: 2, y: 9 }
      }
    ],

    buildings: [],
```

```
    npcs: [],
    objects: []
  }
};
```

未來新增區域時，只需要增加資料，不必大幅修改核心引擎。

第三章：角色移動與場景互動

3.1 玩家移動

沿用目前房間的移動方式。

玩家點擊地板，角色自動尋找最短路徑。

但城市需要新增幾個功能：

功能	規格
點擊移動	點擊可行走地板
BFS 尋路	自動避開建築與障礙物
攝影機	角色移動時跟隨
場景邊界	不可走出地圖
建築碰撞	建築本體不可穿越
入口互動	走到入口附近後切換場景
NPC 互動	點擊 NPC，自動走到附近
物品互動	點擊物品，自動走到互動範圍

我建議保留 BFS，不需要急著改 A*。

目前地圖規模只有 24×18 格，BFS 已經足以處理。

等未來地圖變得非常大，再評估 A* 和分區尋路。

3.2 互動系統

城市中所有互動物件統一使用同一套規則。

例如：

- 點擊 NPC → 走過去 → 開啟對話。
- 點擊咖啡廳 → 走到門口 → 進入咖啡廳。
- 點擊公告欄 → 走過去 → 顯示公告。
- 點擊地面物品 → 走過去 → 拾取物品。
- 點擊工作櫃台 → 開啟工作介面。

建議每個互動物件都有：

JavaScript

```
{
  id: "cafe_counter",
  type: "job_station",
  position: { x: 10, y: 8 },
  interactionRange: 1,
  action: "open_job"
}
```

這樣未來加入新的互動物件時，不需要重新設計點擊系統。

第四章：NPC 系統規格

NPC 是城市生活感的核心。

我不建議一開始就做複雜 AI，先使用固定行為、預設對話和條件式任務。

這樣成本低、容易測試，也不需要呼叫生成式 AI API。

4.1 第一批 NPC 名單

第一版建議五名 NPC。



Mia
咖啡廳老闆

- 咖啡廳打工
- 咖啡豆委託
- 咖啡系列收藏



Leo
麵包師傅

- 麵包店打工
- 食材收集
- 烘焙家具



Nora
超市店員

- 收銀工作
- 商品分類任務



Oliver
公園管理員

- 植物採集
- 公園清潔

- 食物收藏
- 植物圖鑑



Ethan
城市居民

- 尋物委託
- 城市探索
- 隱藏事件

這些是暫定角色名稱和職業，可以之後再做角色個性與完整故事。

4.2 NPC 基本資料

每個 NPC 應有以下欄位：

欄位	說明
id	NPC 唯一識別碼
name	NPC 名稱
areaId	所屬區域
position	初始座標
direction	面向
personality	性格標籤
dialogue	對話資料
jobId	關聯工作
questIds	可提供的任務
schedule	活動時間
affinity	玩家好感度

例如：

JavaScript

```
const npcData = {
  mia: {
    id: "mia",
    name: "Mia",
    areaId: "commercial",
    position: { x: 8, y: 6 },
```

```
    personality: "friendly",

    jobId: "cafe_worker",

    dialogue: {
      greeting: "今天也來幫忙嗎？",
      working: "今天的客人特別多呢！"
    },

    affinityMax: 100
  }
};
```

4.3 NPC 對話系統

我建議對話框採用以下形式：



對話選項分成三類：

1. 一般對話。
2. 接受任務。
3. 查看工作。

不需要每次對話都設計多重選擇分支。

4.4 NPC 好感度

建議好感度採用 0～100。

好感度	稱號	解鎖
0～19	陌生人	基本對話
20～39	認識	新對話、普通委託
40～59	熟人	特殊任務
60～79	朋友	特殊家具
80～100	好朋友	專屬故事、外觀

增加好感度的方式：

- 完成 NPC 委託：+10。
- 每日首次與 NPC 互動：+2。
- 完成特殊劇情：+15。

每日互動增加好感度需要限制，避免玩家重複點擊無限刷取。

第一版可以先不開發好感度，只保留資料欄位。 先讓 NPC 對話和任務正常運作。

第五章：打工系統規格

這是整個城市系統的重點。

你選擇 C：混合型打工。

因此我建議把工作分成三種模式。

類型 A

互動小遊戲型

玩家必須實際操作，完成一段小遊戲才能領取薪水。

例如咖啡廳、收銀員、釣魚。

類型 B

任務工作型

玩家接到工作後，在城市中完成指定任務。

例如送貨、尋找物品、清理公園。

類型 C

簡單工作型

玩家選擇工作、支付少量操作成本或等待遊戲內時間，再領取薪水。

例如幫忙整理貨架、簡單搬運。

5.1 工作種類與規格

第一批建議開發六種工作。

工作	類型	遊戲內容	建議時間
咖啡廳店員	A	製作飲料	2～3 分鐘
超市收銀員	A	掃描、計算、找零	2 分鐘
麵包店助手	A	商品分類、包裝	2 分鐘
外送員	B	城市尋路、送貨	2～4 分鐘
公園清潔員	B	找垃圾、完成清潔	2～3 分鐘
商店整理員	C	選擇工作、領取薪水	1～2 分鐘遊戲內時間

第一階段只做咖啡廳。

其他工作等咖啡廳系統穩定後再開發。

5.2 咖啡廳工作完整玩法



玩家走進咖啡廳，點擊櫃台。

系統顯示：

咖啡廳打工 工作示意

02:00

本次工作剩餘時間

顧客訂單



熱拿鐵
咖啡 × 1、牛奶 × 1

製作流程

1. 選擇咖啡。
2. 加入牛奶。
3. 選擇正確杯子。
4. 交付顧客。

正確訂單
3 / 4

預估收入 **100 金幣**

咖啡廳評分公式

建議：

$$Score = 0.7A + 0.2T + 0.1C$$

- A：訂單正確率。
- T：時間完成率。
- C：連續正確完成訂單比例。

三項都以 0～100 計算。

收入公式：

$$Reward = B \times (0.6 + 0.4 \times Score/100)$$

其中：

- B：基本薪資，100 金幣。
- Score：本次工作分數。

這樣工作分數為 100 時獲得 100 金幣，分數為 50 時獲得 80 金幣。

不建議完全失敗就一毛錢都沒有，可以保留最低基本報酬，避免休閒玩家挫折。

5.3 打工限制

項目	建議規則
每次工作	1～4 分鐘
每種工作	每日最多 3 次領取完整薪水
工作熟練度	完成工作增加
工作獎勵	基本薪資＋表現獎勵
工作失敗	取得部分薪資
重複工作	可繼續遊玩，但超過上限不再取得完整獎勵

這個限制主要是為了控制經濟平衡，不是為了強迫玩家等待。

如果玩家想繼續玩咖啡廳小遊戲，可以繼續，但不應該無限刷金幣。

5.4 工作熟練度

每種工作各自有等級：

等級	熟練度需求	解鎖
Lv.1	0	基礎工作
Lv.2	100	新訂單
Lv.3	300	進階工作
Lv.4	600	特殊訂單
Lv.5	1,000	專屬稱號

工作等級不直接提高薪水太多，主要增加玩法變化和收藏內容。

第六章：任務系統

任務是串聯城市、NPC、打工、探索的核心。

我建議採用資料驅動方式，任務內容放在 JSON 或 JavaScript 資料檔中。

6.1 任務種類

任務類型	說明	例子
主線任務	引導玩家認識城市	認識 Mia
每日任務	每天重置	完成一次打工
每週任務	長期挑戰	完成 10 次工作
NPC 委託	NPC 提供	幫忙尋找咖啡豆
探索任務	城市探索	找到 5 個隱藏物品
收藏任務	收集指定物品	集滿 10 種植物

6.2 第一批主線任務

我建議遊戲開始時不要讓玩家直接面對整座城市。

先安排一段約 10～15 分鐘的新手任務。

任務	內容	獎勵
1. 歡迎來到城市	走到中央廣場	30 金幣
2. 認識新朋友	與 Mia 對話	30 金幣
3. 第一份工作	完成咖啡廳打工	80 金幣
4. 城市小幫手	幫 Oliver 清理公園	50 金幣
5. 回家看看	返回自己的房間	30 金幣
6. 打造新生活	購買一件家具	特殊裝飾

這個流程可以讓玩家在一開始就體驗城市、NPC、工作和房間四個核心系統。

6.3 NPC 委託範例

JavaScript

```
const questData = {
  mia_coffee_beans: {
    id: "mia_coffee_beans",
    title: "咖啡豆不夠了",

    giver: "mia",

    description:
      "Mia 需要 5 個咖啡豆，請幫她收集。",

    objectives: [
      {
        type: "collect",
        itemId: "coffee_bean",
        amount: 5
      }
    ],

    rewards: {
      coins: 120,
      affinity: 10
    },

    repeatable: false
  }
};
```

任務目標建議統一使用事件系統。

例如玩家採集物品時，遊戲發出 `ITEM_COLLECTED` 事件；任務系統收到後，自動更新所有符合條件的任務。

這比在每個採集功能中都寫一遍任務判斷更容易維護。

第七章：城市探索與收藏系統

城市不能只有工作，否則玩家會覺得自己只是每天上班。

我希望玩家在沒有任務的時候，也會想四處走走。

7.1 城市探索互動

第一版先加入三種互動：

物件	互動	獎勵

路邊閃光物品	撿拾	金幣、紀念品
公園植物	採集	植物材料
隱藏物件	點擊調查	特殊收藏品

例如玩家經過一棵樹，發現樹下有一個閃光物品。

點擊後角色走過去，出現：

「你發現了一張舊明信片！」

然後加入收藏圖鑑。

這種設計成本不高，卻可以讓城市探索有驚喜。

7.2 收藏品種類

第一版先規劃四個系列：

系列	收藏品
城市紀念品	明信片、徽章、舊照片
自然收藏	花朵、葉子、蘑菇
食物收藏	咖啡、蛋糕、麵包
隱藏收藏	特殊道具、稀有物件

後續再加入昆蟲、魚類和更多收藏系列。

7.3 收藏品取得機制

不同收藏品應該有不同取得來源。

例如：

- 咖啡豆：咖啡廳任務。
- 花朵：城市公園。
- 明信片：城市探索事件。
- 稀有魚：海濱區釣魚。
- 特殊徽章：NPC 好感度獎勵。

這樣玩家不會只需要待在一個地方就能收集所有東西。

第八章：經濟系統與數值規格

這裡必須重新調整原本 v0.1 的經濟系統。

原本是透過舒適度取得每日金幣，現在新增打工後，打工將成為主要收入來源。

我不建議把原本每日登入收入直接刪掉，而是降低其比重。

8.1 貨幣種類

初期只使用兩種：

貨幣	取得方式	用途
Coins 金幣	打工、任務、探索、登入	家具、一般服裝、日常消費
Materials 材料	採集、任務、探索	家具製作、收藏

未來付費外觀才加入 Gems 星鑽。

不要在城市 MVP 階段同時開發三套貨幣系統。

8.2 金幣收入

以下為建議初始數值：

來源	獎勵
每日登入	30
咖啡廳工作	80～120
麵包店工作	70～110
超市工作	80～130
外送工作	100～160
公園工作	60～100
一般 NPC 委託	50～150
特殊 NPC 委託	150～300
探索事件	5～30

這些是設計目標，必須透過實際測試調整。

8.3 金幣消費

消費項目	建議價格
普通家具	50～100
精緻家具	120～250
稀有家具	300～500
收藏家具	600～1,000
普通服裝	100～250
特殊服裝	300～600
家具製作	需要材料＋少量金幣

目前商店 25～220 金幣的商品不必全部重新定價。

應該先保留舊商品，再逐漸增加新商品價格區間。

8.4 經濟平衡目標

我建議以每天玩 20～40 分鐘的玩家作為測試對象。

假設一個玩家每天：

- 完成兩次工作：200 金幣。
- 完成兩個任務：100 金幣。
- 探索取得：30 金幣。
- 每日登入：30 金幣。

則每日收入：

$$200 + 100 + 30 + 30 = 360$$

這表示玩家平均每天能取得約 360 金幣。

依照這個收入：

- 普通家具可以每天買幾件。
- 精緻家具需要約 1 天累積。
- 稀有家具需要數天。
- 收藏家具則作為長期目標。

這只是初期經濟模型，後續應觀察玩家金幣存量與消費率，避免金幣越累積越多。

8.5 防止刷金幣

必須有以下限制：

1. 每個工作有每日獎勵上限。

2. 每個任務有領取狀態。
3. 重複完成任務不重複發獎。
4. 玩家不能直接修改金幣數量。
5. 每筆獎勵必須有唯一交易 ID。
6. 重要經濟資料由可信任後端驗證。

目前 v0.1 沒有真正帳號系統，因此正式多人參觀與跨裝置存檔的架構還不足以安全處理付費與大量金幣交易。

建議先把城市經濟做成本機可玩的原型，再加入 Authentication 與 Cloud Functions 等可信任後端驗證。

第九章：Firebase 與資料結構

目前你使用 Firebase Realtime Database REST，這點我建議繼續保留。

但新增系統時要避免每次移動角色就寫入雲端。

角色移動、NPC 走動、工作動畫等即時資料都應該留在客戶端。

只有真正需要保存的進度才寫入 Firebase。

9.1 建議資料架構

純文字

```
nook/
|
├── rooms/
|   └── <playerId>/
|
├── idx/
|
├── likes/
|
├── profiles/
|   └── <playerId>/
|
├── playerData/
|   └── <playerId>/
|       ├── economy/
|       |   ├── coins
|       |   └── materials
|       |
|       ├── inventory/
|       |
|       ├── jobs/
|       |   ├── cafe/
|       |   ├── bakery/
|       |   └── cashier/
|       |
|       ├── quests/
|       |
|       ├── collections/
|       |
|       ├── relationships/
```

9.2 資料存取原則

資料	儲存位置	同步方式
玩家位置	本機	場景離開時存檔
NPC 移動位置	本機	不需要同步
金幣	玩家資料	獎勵交易後更新
背包	玩家資料	物品變動後更新
工作進度	玩家資料	完成工作後更新
任務進度	玩家資料	任務事件後更新

收藏圖鑑	玩家資料	解鎖時更新
城市地圖	靜態設定	所有玩家共用
NPC 固定資料	靜態設定	所有玩家共用

城市地圖、建築、NPC 對話等共用資料應該直接放在程式碼或靜態 JSON，不必每個玩家都從 Firebase 重複下載。

9.3 雲端讀寫優化

我建議：

- 進入遊戲時讀取一次玩家資料。
- 遊戲期間先更新本機 State。
- 重要操作完成後批次同步。
- 避免角色每走一步就寫入 Firebase。
- 避免每個 NPC 對話都產生雲端請求。
- 房間參觀時才讀取其他玩家的房間資料。

這樣可以繼續維持低成本架構。

第十章：程式架構與模組規劃

目前簡報的程式架構是：

- `raster.js`
- `models.js`
- `scene.js`
- `game.js`
- `cloud.js`

我建議保留原有繪圖核心，新增城市模組。

10.1 正式目標結構

純文字

```
js/
|
├── core/
|   ├── game.js
|   ├── state.js
|   ├── scene-manager.js
|   ├── event-bus.js
|   └── input.js
|
├── rendering/
|   ├── raster.js
|   ├── models.js
|   ├── scene.js
|   └── camera.js
|
├── city/
|   ├── city-map.js
|   ├── city-scene.js
|   ├── city-area.js
|   ├── building.js
|   ├── city-object.js
|   └── collision.js
|
├── characters/
|   ├── player.js
|   ├── npc.js
|   └── npc-schedule.js
|
├── systems/
|   ├── interaction.js
```

10.2 必須新增的核心模組

模組	職責
SceneManager	管理城市、房間、建築內部場景
Camera	處理城市攝影機
CityMap	地圖格子、建築、障礙物
Interaction	統一互動邏輯
NPC	NPC 位置、對話、行為
EventBus	系統之間傳遞事件
JobSystem	管理工作和薪資
QuestSystem	任務進度
EconomySystem	金幣與材料
InventorySystem	玩家物品
CollectionSystem	收藏圖鑑

其中 `EventBus` 很重要。

例如：

玩家完成咖啡廳工作：

純文字

```
JOB_COMPLETED
      |
      ├── EconomySystem：增加金幣
      |
      ├── QuestSystem：更新任務
      |
      ├── NPCSystem：增加好感度
      |
      ├── AchievementSystem：更新成就
      |
      └── SaveSystem：儲存進度
```

這樣各個系統不用直接互相依賴。

第十一章：正式開發路線圖

我重新安排了開發順序。

這次最重要的是：**先讓玩家能夠在城市走動，再讓城市裡的人和建築有用途，最後才增加豐富的內容。**

不然很容易出現一張漂亮的城市地圖，卻不知道進去要做什麼。

建議總工期：12～16 週

以一人開發、Claude Code 協助為前提，這是規劃估算，不是保證工期。

Phase 0：原有遊戲測試

預估 1 週

工作	驗收標準
測試現有房間	家具放置、旋轉正常
測試角色	四方向移動正常
測試存檔	重新整理後資料保留
測試參觀	可以正常讀取別人房間
備份程式碼	建立可回復的版本

Phase 1：城市引擎

第一個開發重點

預估 2～3 週

功能	驗收標準
CityScene	可以進入城市
CityMap	可以載入格子地圖
Camera	角色移動時攝影機跟隨
BFS	可以繞過建築
Building	建築可以放置
SceneManager	可以切換區域
Exit	可以從一區移動到另一區
Collision	不可以穿越建築

完成標準：

玩家可以從自己的房間出門，在中央廣場和商業街之間移動，並且可以進入咖啡廳。

此階段不需要 NPC、工作或任務。

Phase 2：NPC 與對話

預估 1～2 週

功能	驗收標準
NPC 顯示	可以在城市中看見 NPC
NPC 移動	可以在指定範圍走動
NPC 點擊	玩家自動走到附近
對話框	正確顯示名稱與對話
對話選項	可以選擇接受、拒絕
NPC 資料	可新增不同 NPC

完成標準：

玩家可以在城市遇到 Mia，和她交談，並且看到工作選項。

Phase 3：咖啡廳打工

第一個完整遊戲玩法

預估 2 週

功能	驗收標準
工作入口	從 NPC 或櫃台進入
訂單系統	隨機產生訂單
製作流程	可以正確選擇材料
計分系統	正確計算分數
薪水	完成後獲得金幣
工作紀錄	正確保存工作次數
熟練度	工作後增加經驗

完成標準：

玩家可以完成一次完整咖啡廳工作，並把賺到的金幣帶回房間購買家具。

這是第一個真正值得邀請朋友試玩的版本。

Phase 4：任務與經濟

預估 2 週

功能	驗收標準
每日任務	正確重置
NPC 委託	接受、進行、完成
任務獎勵	不可重複領取
金幣系統	收入與消費正常
背包	可以查看材料
存檔	重新進入仍保留進度

完成標準：

玩家能夠透過工作與任務賺錢，並使用收入購買家具。

Phase 5：城市探索與收藏

預估 2 週

功能	驗收標準
城市互動物件	可以調查與採集
隨機事件	符合條件時出現
物品取得	正確加入背包
收藏圖鑑	顯示已解鎖項目
探索任務	可以追蹤進度

完成標準：

玩家不需要打工，也能透過探索獲得物品、完成目標。

Phase 6：城市擴充

預估 2～3 週

功能	驗收標準
麵包店	第二種工作
超市	第三種工作
公園	採集與清潔
外送	城市任務型工作
新 NPC	對話與任務正常
新區域	分區切換正常

這個階段再考慮住宅區、海濱區和工坊區。

Phase 7：營利與正式測試

預估 2 週以上

功能	驗收標準
存檔轉移	跨裝置恢復
帳號	可以登入與同步
社群	房間參觀正常
廣告	完整觀看才發獎
商城	外觀商品交易正常

防刷	無法任意修改金幣
效能	手機操作順暢

廣告及付費商品可以晚於 v1.0 推出，不要讓營利系統拖慢城市玩法的完成。

第十二章：每個階段的品質檢查

我希望你每次讓 Claude Code 開發完，都能用以下標準檢查。

測試項目	通過條件
場景切換	不會黑畫面或卡死
玩家移動	不會穿牆或卡在建築
NPC 互動	不會重複開啟對話
工作獎勵	不會重複領取
任務進度	不會錯誤累加
金幣	不會出現負數
背包	不會重複生成物品
存檔	重新整理後資料保留
手機	橫向操作正常
原有房間	既有家具功能不受影響

另外，因為原本是 HTML5 Canvas，新增城市後一定要測試：

- 低階手機效能。
- 地圖移動時的畫面更新。
- 角色與建築的層級排序。
- 多個 NPC 同時存在時的效能。
- 長時間遊玩後是否有記憶體持續增加。

第十三章：交給 Claude Code 的第一階段開發指令

我不建議現在就把整份規格書一次丟給 Claude，要求它全部實作。

你可以先把上面的規格書保存為 `PIXELNOOK_CITY_SPEC.md`，放進專案根目錄。

接著先讓 Claude 做城市引擎。

以下是我幫你準備的第一個開發指令。

PixelNook Phase 1：城市引擎開發任務

請先閱讀專案根目錄的 PIXELNOOK_CITY_SPEC.md，以及目前所有 JavaScript 程式碼。

本次只實作 Phase 1：城市地圖、角色移動、攝影機及場景切換。

一、目前技術架構

- HTML5 Canvas
- 原生 JavaScript
- 64×32 px 等角格子
- 800×480 邏輯畫布
- 四方向角色移動
- 現有 BFS 尋路
- Firebase Realtime Database REST
- GitHub Pages

二、開發目標

建立第一個可玩的城市場景。

城市包含：

1. 中央廣場
2. 商業街
3. 咖啡廳室內場景

三、功能需求

1. CityScene

建立獨立城市場景，不要破壞現有房間場景。

2. CityMap

- 使用 24×18 格地圖。
- 格子維持 64×32 px。
- 支援地板、道路、草地、建築及障礙物。
- 地圖資料與繪圖邏輯分離。

3. Player Movement

- 沿用現有四方向角色模型。
- 點擊地板自動移動。
- 使用 BFS 尋路。
- 不可穿越建築及障礙物。

4. Camera

- 角色移動時攝影機跟隨。
- 畫面固定 800×480 邏輯解析度。
- 不可因攝影機移動造成像素模糊。

5. SceneManager

- 支援房間、城市、建築室內場景。
- 支援場景切換。
- 切換時正確設定玩家出生位置。
- 記錄各場景的玩家位置。

6. 建築與入口

- 中央廣場可以前往商業街。
- 商業街可以進入咖啡廳。
- 建築碰撞範圍正確。
- 入口附近可以互動。

四、美術規則

- 全部使用程式繪圖。
- 不購買、不下載外部素材。
- 保持目前 PixelNook 的等角像素風格。
- 沿用既有角色比例及四方向繪製。
- 不使用漸層或模糊效果。

五、重要限制

1. 不可重寫 raster.js 的繪圖核心。
2. 不可破壞 models.js 的角色模型。
3. 不可刪除現有家具與房間功能。
4. 不可在本階段加入 NPC、任務、工作或經濟系統。
5. 不可擅自修改 Firebase 資料結構。
6. 優先新增獨立模組。
7. 如果現有程式架構不適合，先提出修改方案，不要直接大規模重構。

六、開發流程

第一步：先閱讀程式碼，整理目前場景、尋路及繪圖架構。

第二步：提出預計修改的檔案清單。

第三步：說明如何讓城市與房間共用角色及繪圖核心。

第四步：開始實作。

第五步：執行測試。

第六步：回報：

- 新增檔案
- 修改檔案
- 完成功能
- 測試結果
- 尚未解決的問題

本次以完成一個可自由移動、可以切換場景的城市原型為唯一目標。

第十四章：我建議你額外注意的三個設計問題

最後，有三件事我希望在正式開發時不要忽略。

1. 不要讓工作變成唯一賺錢方式。

工作是城市經濟的主要來源，但探索、NPC 委託、收藏活動也應該能取得收入。

否則玩家會覺得每天都必須上班。

2. 不要讓城市只有 NPC 站著。

即使不做即時多人，也可以讓 NPC 在不同位置活動。

例如 Mia 在咖啡廳、Oliver 在公園，晚上 Ethan 會出現在廣場。

這些簡單行為就能增加城市的生命感。

3. 不要讓每個功能都需要雲端運算。

城市移動、工作小遊戲、NPC 行為、物品生成都可以在本機執行。

只有玩家進度、房間資料、重要經濟交易需要同步。

這才符合你一人開發、低成本營運的目標。

最終產品藍圖

我把目前 PixelNook 的發展方向濃縮成這張表：

系統	v1.0 必要程度	長期擴充
私人房間	必要	多房間、擴建
等角城市	必要	更多城市區域
NPC 對話	必要	好感度、人物故事
打工	必要	更多職業
任務	必要	主線、每日、每週
探索	必要	隨機事件、隱藏區域
收藏	必要	魚類、昆蟲、紀念品
家具製作	可延後	完整製作系統
房間社群	必要	主題競賽、留言

寵物	可延後	養成、互動
天氣	可延後	季節、節日
好友聊天	不建議初期	視需求評估

我對 PixelNook 最終的定位是：

> 一款以城市生活為核心的單人像素模擬遊戲。玩家可以在城市中工作、探索、認識居民、完成委託、收集物品，透過自己的努力賺取收入，再回到私人房間打造屬於自己的生活空間。玩家也能非同步參觀其他人的房間，分享自己的生活成果。

我認為這個方向已經足以支撐一款完整的休閒手機遊戲，而且不必依賴即時多人連線。

下一個里程碑不要設定成「完成整座城市」，而是設定成「玩家可以在中央廣場走到咖啡廳，遇見 Mia，完成一份工作，賺到錢，再回家買家具」。

只要這條流程真正好玩，PixelNook 就有了可以繼續擴大的核心。